Midterm Report

Topic: Visual Drilling Operations & Predictive Maintenance System

Student Name: Uchechi Sebastian

Student ID: 300393092

Course: CSIS 4495 002 Applied Research Project

Section: 002

Team Lead: Uchechi Sebastian (Individual Project)

Project Demo Video

Table of Contents

Introduction ... 4
1.1 Problem Statement ... 4
1.2 Literature and Related Work ... 5
A. Real-time drilling data visualization ... 5
B. Wellbore schematics and trajectory visualization ... 5
C. Drilling optimization and predictive analytics ... 5
E. Non-productive time (NPT) analysis ... 6
1.3 Knowledge Gaps and Limitations ... 6
Gap 1 — Lack of depth-based event visualization ... 6
Gap 2 — Limited integration of drilling events with maintenance insights ... 6
Gap 3 — Lack of explainability in predictive systems ... 6
Gap 4 — Fragmentation across tools ... 6
Gap 5 — Limited accessibility and reusability ... 7
1.4 Hypotheses ... 7
1.5 Assumptions ... 7
1.6 Expected Benefits ... 8
Summary of the Initially Proposed Research Project ... 8
2.1 Research Objectives ... 8
2.2 Methodology ... 8
1. Data Preparation: ... 8
2. Backend Development: ... 9
3. Frontend Development: ... 9
4. Analytics and Explainability: ... 9
5. Reporting and Evaluation: ... 9
2.3 Data Collection ... 9
2.4 Sample Size ... 9
2.5 Data Analysis Techniques ... 10
2.6 Research Procedures ... 10

2.6 Technologies to be Used: 10
2.7 Expected Results 11
Changes to the Proposal 11
3.1 Change of Database Technology: PostgreSQL/MySQL → SQLite 11
Project Planning and Timeline 12
4.1 Project Timeline 12
4.2 Gantt chat – Project timeline 13
Implemented Features 14
5.1 Home screen: 14
5.2 The sign-up page. 15
5.3 The login page. 15
5.4 Wells Page 16
5.5 Create New Well 17
5.6 Drilling Dashboard 18
5.7 Segment Details Panel 19
5.8 Predictive Maintenance Page 21
5.9 Drilling Report Page 23
5.10 Daily Reports Page 24
5.11 Extracted Operations Page 25
5.12 Extracted Operations Page 26
AI Use Section 27
6.1 AI use Table 27
Work Date / Hours Log 29
Conclusion 37
References: 38
Literature references 38
Appendix 39
AI Prompt(s) Used 39

Introduction

Drilling operations in the oil and gas industry are complex, data-intensive processes that generate large volumes of operational information throughout the lifecycle of a well. During drilling, data is continuously recorded through daily drilling reports, operation logs, and event summaries, capturing information such as depth progression, operational activities, downtime events, and equipment involvement. This data plays a critical role in post-operation analysis, operational learning, and planning for future drilling campaigns.

Despite the availability of such data, interpreting drilling operations remains a challenging task. Drilling engineers often rely on textual reports, spreadsheets, and static dashboards to review what occurred during drilling. These representations can make it difficult to quickly understand how operational problems relate to specific depth intervals, how issues evolve, and how repeated events contribute to increasing operational risk. As a result, valuable insights into non-productive time, recurring operational issues, and potential maintenance concerns may not be immediately visible.

Within this context, there is a growing need for software tools that not only display drilling data but also help interpret operational patterns and provide explainable insights that support decision-making during and after drilling operations.

1.1 Problem Statement

Drilling operations produce detailed operational records that describe what occurred during the drilling process; however, transforming these records into an actionable understanding remains a significant challenge. Engineers must interpret large volumes of operational data to determine where drilling problems occurred, how these problems evolved, and what operational behaviors may have contributed to increasing risk or non-productive time. When this interpretation is inefficient or unclear, valuable lessons from drilling operations may be lost.

This project is framed around the following research questions:

1. How can drilling operation data be visually organized to clearly show where operational problems occur along well depth?

2. How can recorded operational events be analyzed to identify emerging patterns that indicate increasing operational or maintenance risk?
3. How can analytics be presented in an explainable way that supports engineering judgment rather than replacing it?

These questions are important because current drilling analysis tools often emphasize data reporting rather than interpretation, making it difficult to quickly recognize patterns and understand their implications. Addressing these questions can improve post-drilling analysis, support operational learning, and contribute to more informed decision-making in future drilling operations.

1.2 Literature and Related Work

A. Real-time drilling data visualization

Commercial platforms such as Petrolink and Halliburton HalVue focus on real-time monitoring, providing dashboards for torque, weight on bit, ROP, and rig activity. These systems emphasize operational awareness and data streaming but do not provide depth-based event visualization or post-operation analysis.

B. Wellbore schematics and trajectory visualization

Tools like wellVizion offer wellbore schematics, trajectory plots, and anti-collision visualization. These systems are valuable for planning and directional drilling, but do not integrate operational events (e.g., stuck pipe, reaming, NPT) along depth.

C. Drilling optimization and predictive analytics

Platforms such as Corva and SLB DrillOps incorporate predictive analytics, real-time optimization, and automated alerts. Their focus is on improving drilling efficiency during operations using machine learning models. However, these systems are typically enterprise-scale, proprietary, and optimized for real-time rig data rather than post-run event analysis.

E. Non-productive time (NPT) analysis

NPT is widely studied as a key performance indicator in drilling operations. Research focuses on the classification of NPT events, root-cause analysis, and cost impact. However, NPT is typically analyzed in tabular or report form, not through interactive depth-based visualization.

Overall, the literature shows strong development in real-time monitoring, trajectory visualization, and predictive analytics - but these areas remain siloed and do not converge into a unified, explainable, depth-based decision-support tool.

1.3 Knowledge Gaps and Limitations

Despite the availability of advanced drilling software, several gaps remain unaddressed:

Gap 1 — Lack of depth-based event visualization

No major tool provides a simple, interactive vertical wellbore view where drilling events are mapped as **color-coded segments along depth**. Existing systems focus on real-time curves, not post-operation event visualization.

Gap 2 — Limited integration of drilling events with maintenance insights

Current predictive maintenance tools rely on sensor data or machine-learning models but do not incorporate **event history** (e.g., stuck pipe, reaming frequency, NPT clusters) as indicators of equipment stress or operational risk.

Gap 3 — Lack of explainability in predictive systems

Most commercial predictive tools operate as black boxes. Engineers often cannot see **why** a risk score increased.
Your system provides **transparent, explainable logic** tied directly to recorded events.

Gap 4 — Fragmentation across tools

Drilling visualization, KPI analysis, and maintenance prediction exist in separate platforms.
There is no lightweight, unified, web-based tool that combines: Drilling KPIs, depth-based event visualization, Predictive maintenance indicators, and Explainable insights.

Gap 5 — Limited accessibility and reusability

Enterprise tools are expensive, proprietary, and require real-time rig data. This system is:

- Upload-based
- Reusable across wells
- Suitable for post-run analysis
- Built with open technologies

This makes it accessible for training, engineering studies, and smaller operators.

This project addresses these gaps by developing a full-stack, reusable decision-support application that (1) visualizes drilling operations and events along a vertical wellbore (pipe) depth view, (2) computes key KPIs such as NPT and event frequency, (3) provides explainable risk indicators based strictly on recorded operational events, and (4) supports practical workflow needs such as well selection via a location map and report export.

1.4 Hypotheses

1. Depth-based visualization of drilling events will improve engineers' ability to identify operational risks compared to traditional tabular or curve-based reports.
2. Event-driven predictive maintenance indicators can highlight equipment stress earlier than sensor-only approaches.
3. Explainable insights increase user trust and adoption compared to black-box predictive models.
4. A unified, web-based platform will reduce analysis time by consolidating drilling KPIs, events, and maintenance indicators in one interface.

1.5 Assumptions

1. Drilling event data (e.g., stuck pipe, reaming, NPT) is available in structured or semi-structured form.
2. Depth information is recorded consistently across drilling operations.
3. Engineers benefit from visual, depth-oriented representations of operational events.
4. Predictive maintenance can be meaningfully informed by event frequency, severity, and clustering.
5. The system is intended for post-operation analysis, not real-time rig control.

1.6 Expected Benefits

1. Faster identification of operational issues through intuitive depth-based visualization.
2. Improved maintenance planning by linking drilling events to equipment stress indicators.
3. Better decision-making through integrated KPIs, event history, and risk scoring.
4. Higher transparency due to explainable logic behind every insight.
5. Reusability across wells and campaigns, supporting continuous improvement.
6. A modern, full-stack engineering tool demonstrates strong technical and analytical capability.

Summary of the Initially Proposed Research Project

2.1 Research Objectives

The main objectives of this project are to:

1. Design a depth-based visualization that clearly represents drilling operations and events along the wellbore.
2. Develop analytics to compute key drilling performance indicators, such as non-productive time (NPT), washout, and event frequency.
3. Implement predictive maintenance indicators that highlight increasing equipment risk based on recorded operational behavior.
4. Provide explainable insights that help users understand why operational risks are flagged.
5. Deliver a reusable, full-stack web application that supports drilling data interpretation and reporting.

2.2 Methodology

The methodology consists of the following steps:

1. Data Preparation:

   Simulated or historical drilling datasets will be structured into standardized formats representing wells, drilling operations, events, and equipment involvement.

2. ### Backend Development:

   A backend service will be implemented using Python and FastAPI to manage data ingestion, storage, and analytics. The backend will compute KPIs, identify recurring patterns, and generate risk indicators based on predefined rules and statistical trends.

3. ### Frontend Development:

   A React-based user interface will be developed to support well selection, depth-based visualization, KPI display, and interaction with drilling events. A vertical wellbore (pipe) visualization will be used to map operational data directly to depth.

4. ### Analytics and Explainability:

   Rule-based and basic statistical methods will be applied to identify emerging operational patterns and maintenance risk. All outputs will be explainable, with clear links to the underlying operational events and data.

5. ### Reporting and Evaluation:

   The system will support the generation of downloadable summary reports. The prototype will be evaluated based on usability, clarity of visualization, and its ability to support the interpretation of drilling operations.

2.3 Data Collection

The system will use **simulated or historical drilling datasets** consisting of:

Well identifiers and total depth, Depth intervals, Drilling operation types, Recorded drilling events (e.g., stuck pipe, downtime), Non-productive time (NPT), and equipment involvement in events.

No real-time sensors or proprietary data will be required.

2.4 Sample Size

The sample dataset will include:

- **Multiple wells** (e.g., 1–10 wells)
- Each well contains **several hundred depth intervals.**
- Dozens of recorded operational events per well

2.5 Data Analysis Techniques

The following data analysis techniques will be applied:

- **Descriptive analytics:**
  Aggregation of drilling events and non-productive time by depth and operation type.
- **KPI computation:**
  Calculation of key indicators such as total NPT, event frequency, and drilling efficiency metrics.
- **Pattern identification:**
  Rule-based and basic statistical analysis to identify recurring events, clustering of issues within depth intervals, and increasing operational or maintenance risk.
- **Explainable logic:**
  All identified risks and alerts will be linked directly to recorded operational data, ensuring transparency and interpretability.

2.6 Research Procedures

The research procedures will follow these steps:

1. Generate and prepare simulated drilling datasets based on realistic operational scenarios.
2. Ingest and store data using a backend service implemented in Python.
3. Apply analytics to compute KPIs and identify operational patterns.
4. Visualize results using a depth-based wellbore interface.
5. Generate downloadable summary reports based on system outputs.
6. Evaluate the system based on usability, clarity of visualization, and interpretability of analytics.

2.6 Technologies to be Used:

- **Operating System / Platform:** Web-based application
- **Programming Languages:** Python, JavaScript
- **Backend Framework:** FastAPI
- **Frontend Framework:** React.js
- **Database:** PostgreSQL or MySQL

- **Visualization:** SVG / D3.js
- **Deployment (Optional):** Docker

2.7 Expected Results

The expected outcome is a **fully functional web application** that:

- Allows users to select wells via a location-based interface.
- Visualizes drilling events along a vertical wellbore.
- Displays drilling KPIs and high-risk zones.
- Provides explainable insights into operational issues.
- Generates downloadable drilling summary reports.

The system is expected to demonstrate practical applicability in the interpretation of drilling data and operational review.

Changes to the Proposal

During the development of the project, only one modification was made to the initial proposal:

3.1 Change of Database Technology: PostgreSQL/MySQL → SQLite

- **Description of Change:**
  The original proposal planned to use a full-scale relational database such as PostgreSQL or MySQL. However, during implementation, this was replaced with **SQLite** as the primary database.
- **Justification:**
  1. **Simplicity and Ease of Setup:** SQLite does not require an external database server, making it much easier and faster to configure during development.
  2. **Reduced Overhead:** Removing the need for server installation, configuration, and management allowed more focus on backend logic, analytics, and visualization features.
  3. **Suitable for Prototyping:** As a file-based database, SQLite is ideal for building prototypes and academic projects where rapid iteration is more important than large-scale deployment.

4. **No Change in Data Structure:** SQLite fully supports the relational schemas originally designed for PostgreSQL/MySQL, so no major redesign was required.

Final Reasoning:

Given the project goals, timeline, and the need for a smooth, self-contained development workflow, SQLite was the most efficient choice. It meets all the data storage requirements of the prototype while simplifying the overall implementation process.

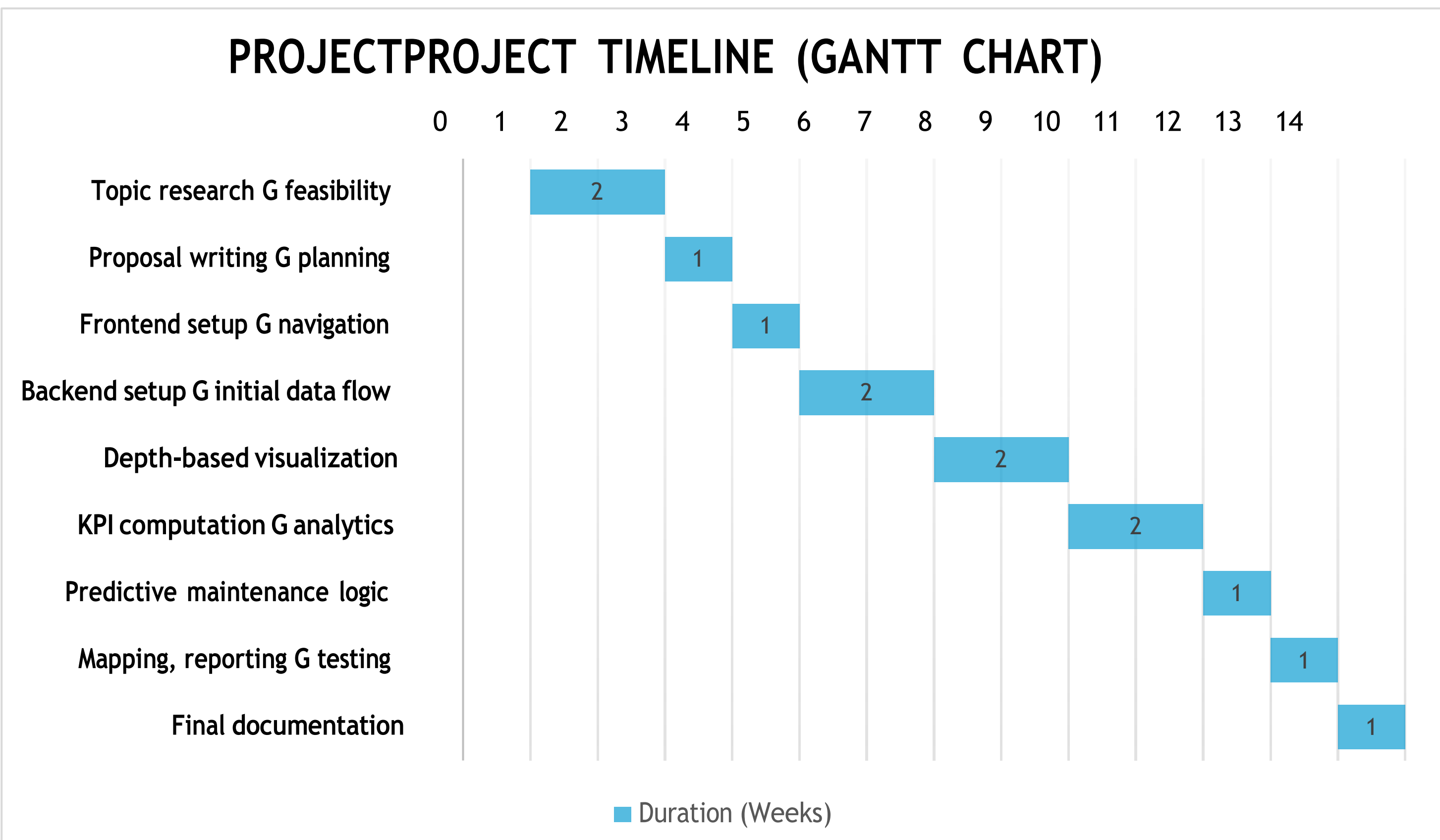
Project Planning and Timeline

4.1 Project Timeline

Phase	Deadline	Milestones	Deliverables
Phase 1: Topic Exploration & Domain Research	Weeks 1 - 2	Explored potential project domains Reviewed drilling operations context Identified problem area and feasibility	Project idea selection Initial domain understanding
Phase 2: Proposal Development & Planning	Week 3	Defined project scope and objectives Identified data requirements Prepared a formal project proposal	Submitted project proposal Finalized research questions and methodology
Phase 3: Frontend Development (Core UI)	Week 4	Implement the main application layout Build well selection and navigation views	Initial React frontend Well selection interface
Phase 4: Backend Setup & Initial Data Flow	Weeks 5–6	Set up backend framework (FastAPI) Designed initial database schema Implemented basic API endpoints Connected frontend to backend using sample data	Functional backend service Initial API endpoints Frontend consuming backend data
Phase 5: Depth-Based Visualization	Weeks 7–8	Implemented vertical wellbore (pipe) visualization Mapped drilling events to depth intervals	Interactive depth-based visualization
Phase 6: KPI Computation & Analytics	Weeks 9–10	Compute NPT and event-based KPIs Aggregate data by depth and operation	KPI analytics module

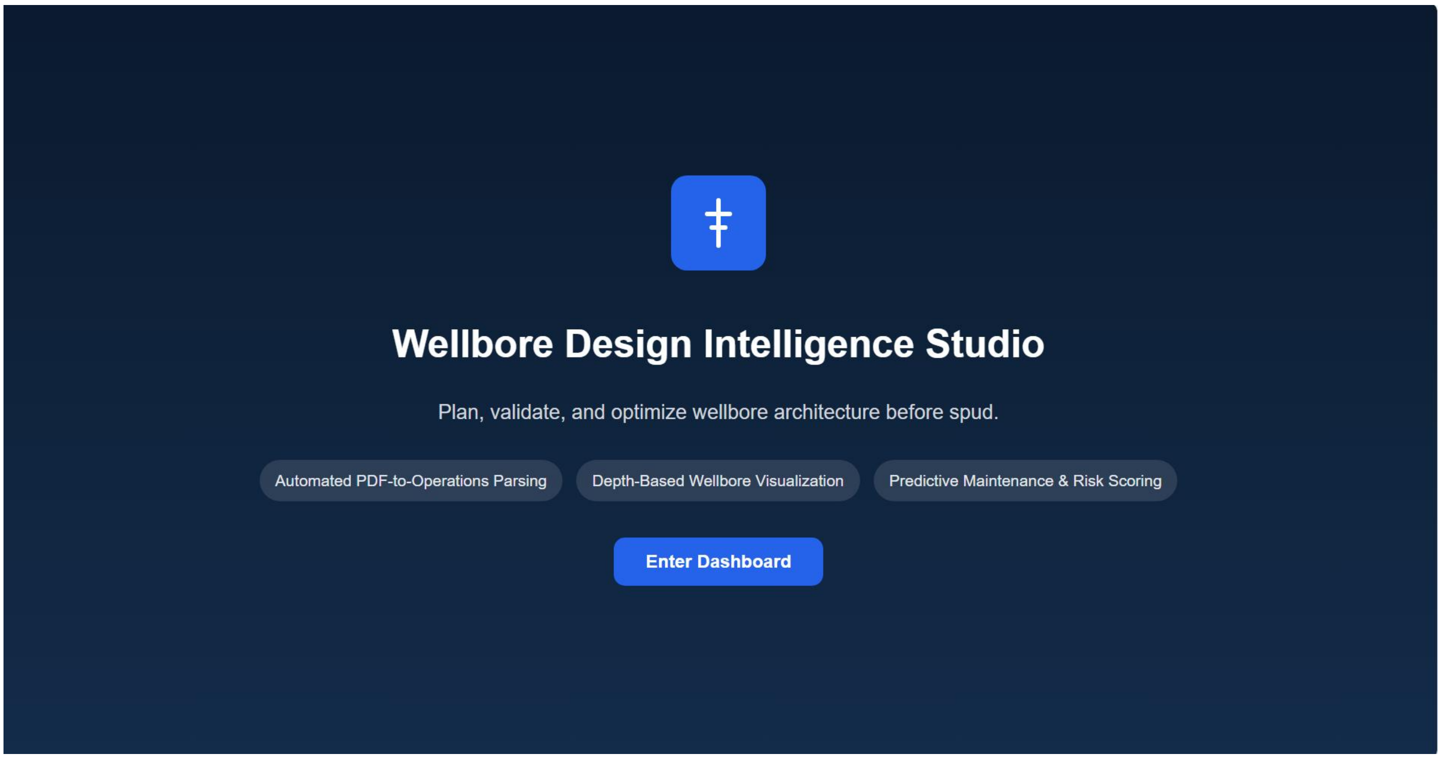
Phase 7: Predictive Maintenance Logic	Weeks 11	Implement rule-based risk indicators Link risk to recorded events and equipment	Predictive maintenance indicators with explanations
Phase 8: Mapping & Report Generation & Testing	Weeks 12	Implement a well location map view Enable downloadable drilling summary reports. Test system functionality.	Location-based well view Report export feature Tested application
Phase 9: Final documentation	Week 13	Prepare final documentation	Final project report

4.2 Gantt chat – Project timeline



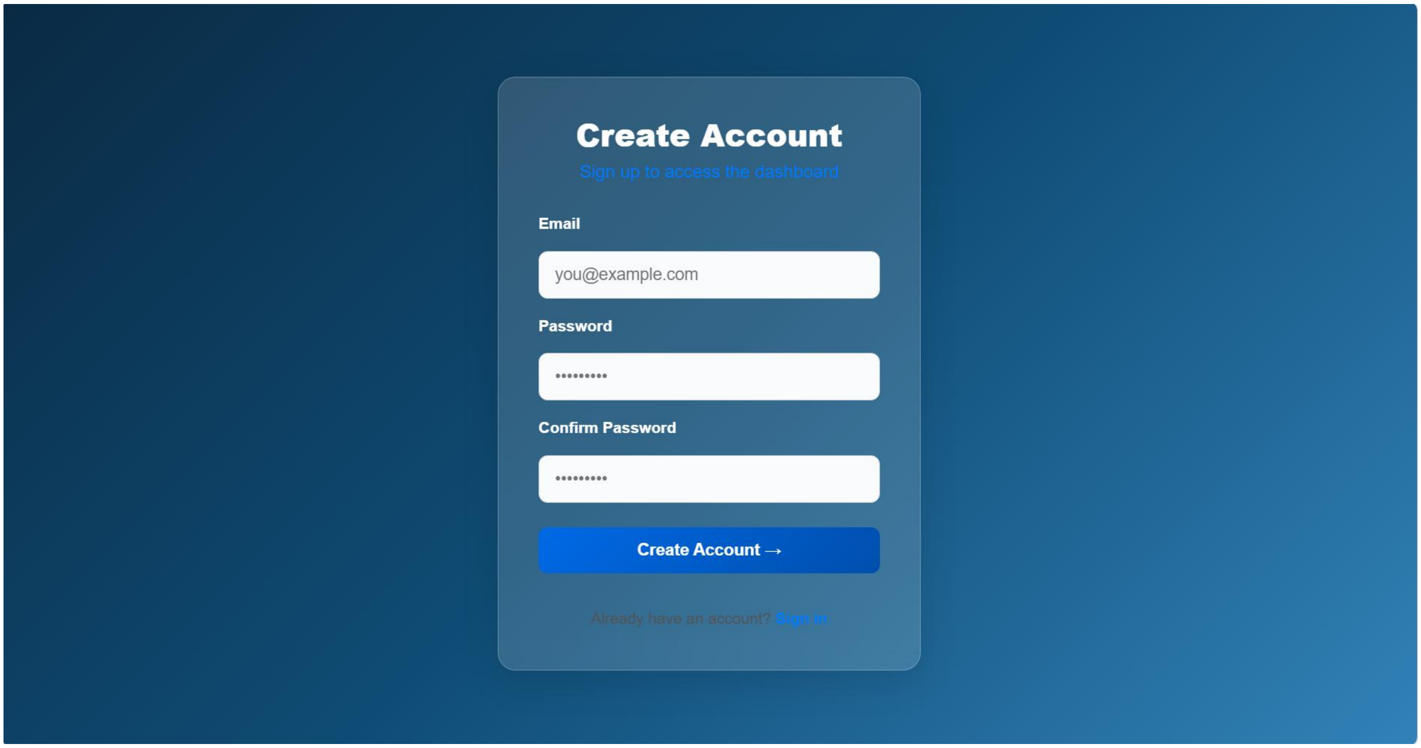
Implemented Features

5.1 Home screen:

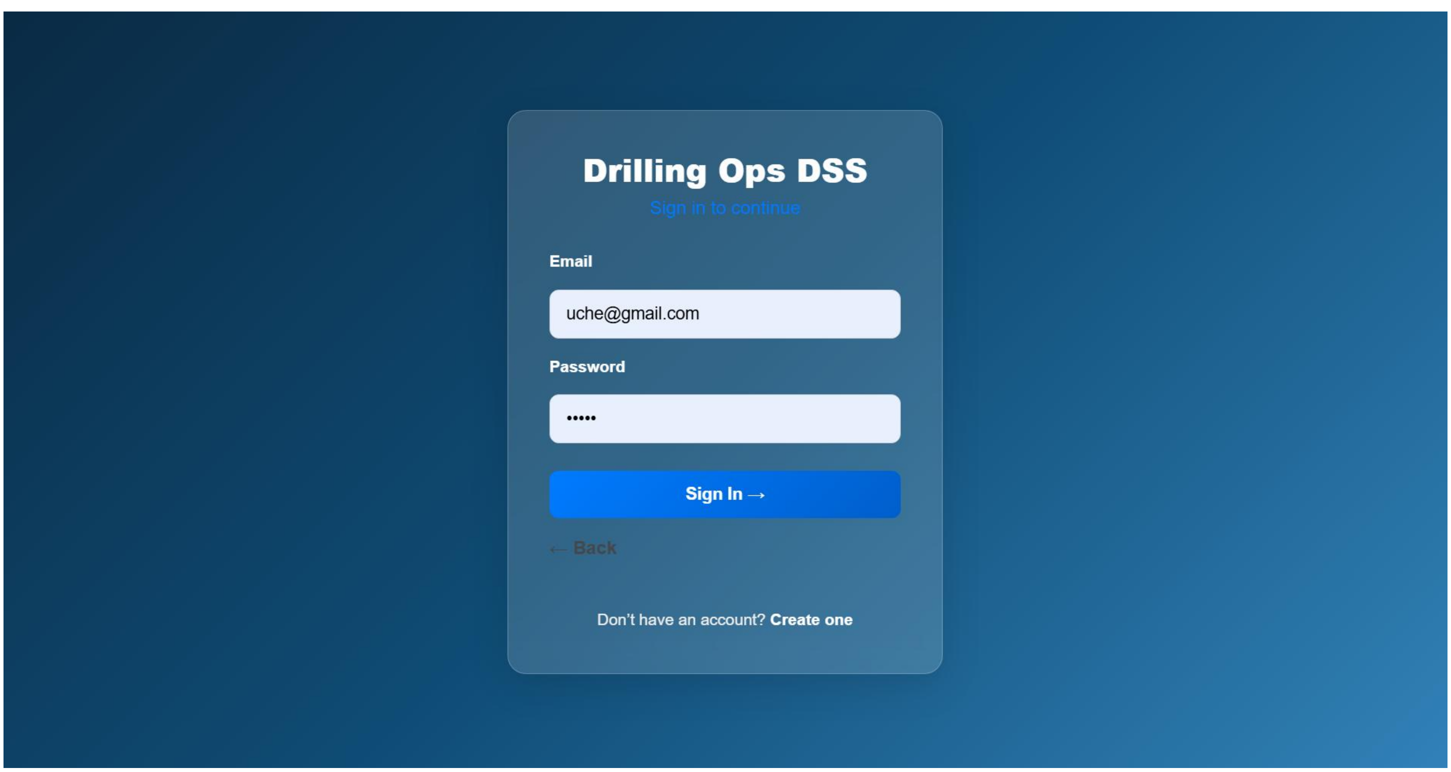


This home screen introduces the Wellbore Design Intelligence Studio, highlighting its purpose in transforming drilling reports into actionable operational insights. It presents the three core capabilities automated PDF parsing, depth-based wellbore visualization, and predictive maintenance to guide users into the main dashboard.

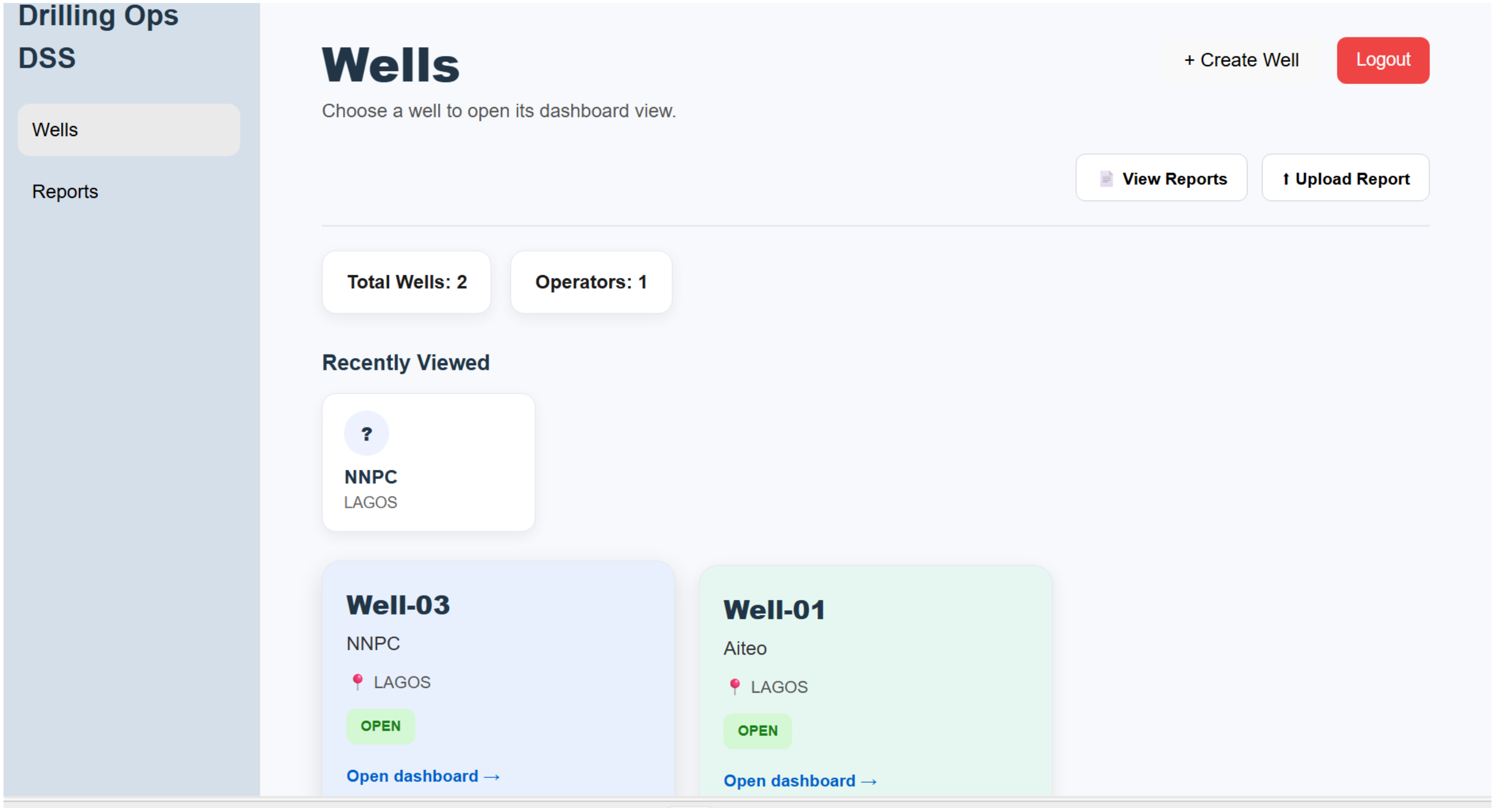
5.2 The sign-up page.



5.3 The login page



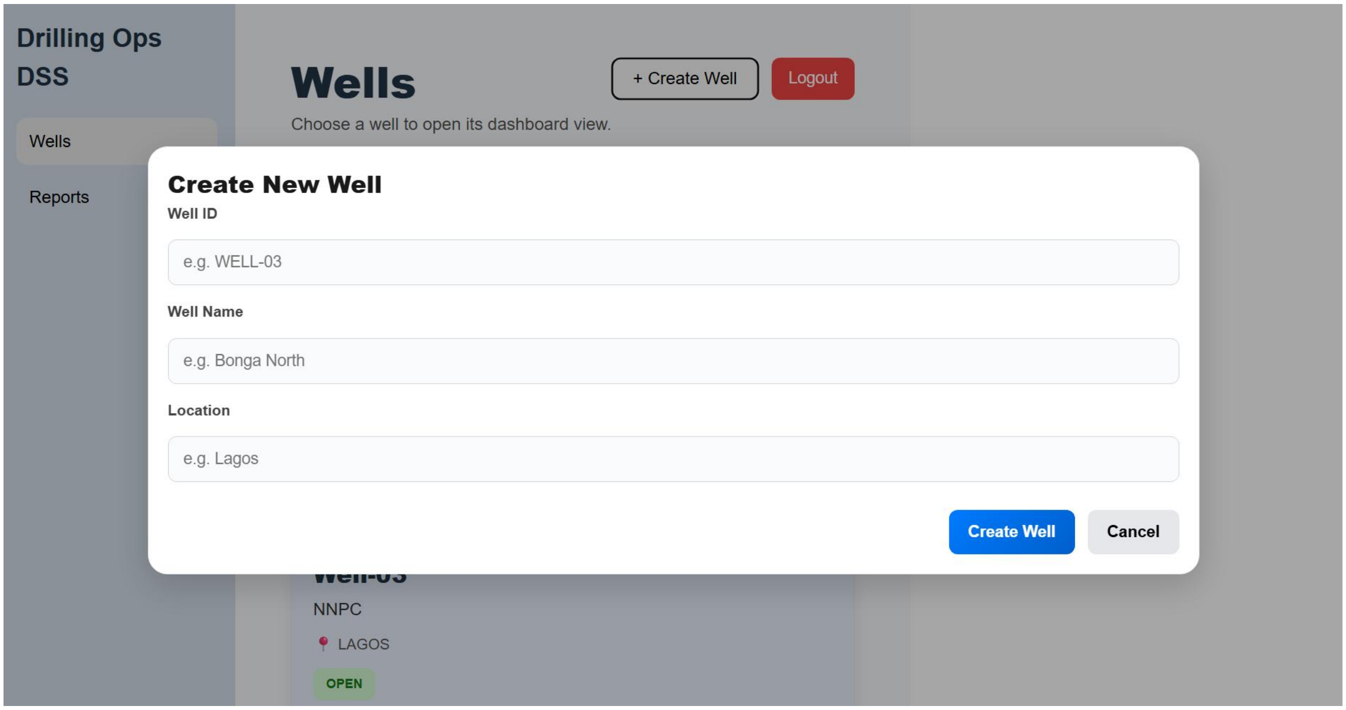
5.4 Wells Page



- **Well Management Overview:** Provides a centralized place to view and manage all wells in the system. Users can quickly scan the list of available wells, their operators, and their locations before diving into detailed analytics.
- **Recently Viewed Wells:** Displays the wells the user interacted with most recently, enabling fast navigation back to active projects without searching through the full list.
- **Create New Well:** A dedicated action button that allows users to register a new well into the system. This supports onboarding new drilling projects and expanding the dataset.
- **Upload Report:** Enables users to upload daily drilling reports (PDFs). These reports are automatically parsed and converted into structured operational data for analysis.
- **View Reports:** Provides direct access to the full library of uploaded reports for the selected well or across all wells, supporting audit trails and historical review.
- **Summary KPIs (Top-Right Cards):** Shows high-level statistics such as:
    - Total Wells: number of wells currently registered
    - Operators: number of unique operating companies in the system These KPIs give users a quick snapshot of system activity.
    - Navigation Sidebar

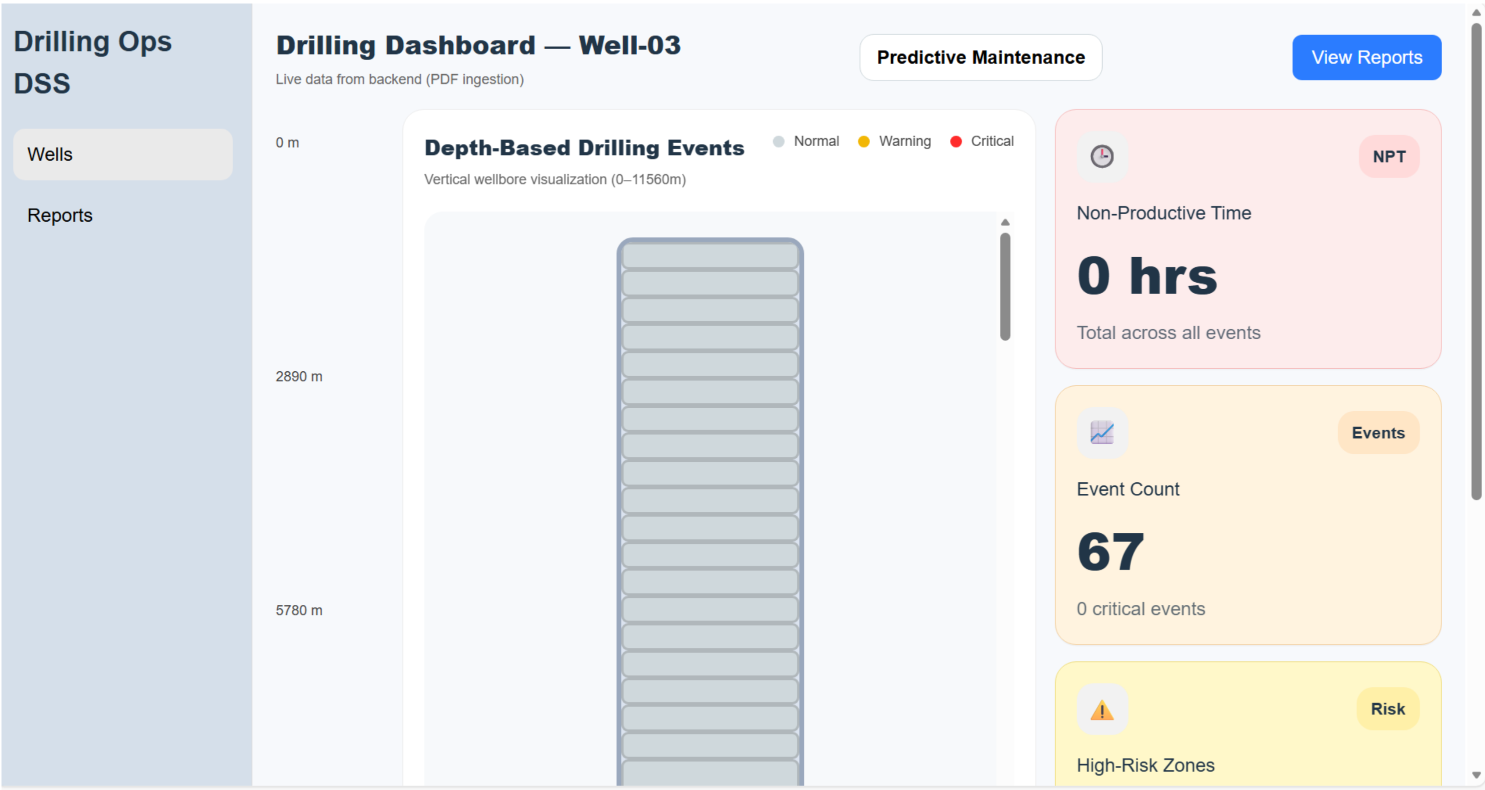
- **A simple, persistent menu offering quick access to:** Wells and reports
- **Logout Control:** Allows users to securely exit the system, maintaining data privacy and session integrity.

5.5 Create New Well



A dedicated action button that allows users to register a new well into the system. This supports onboarding new drilling projects and expanding the dataset.

5.6 Drilling Dashboard

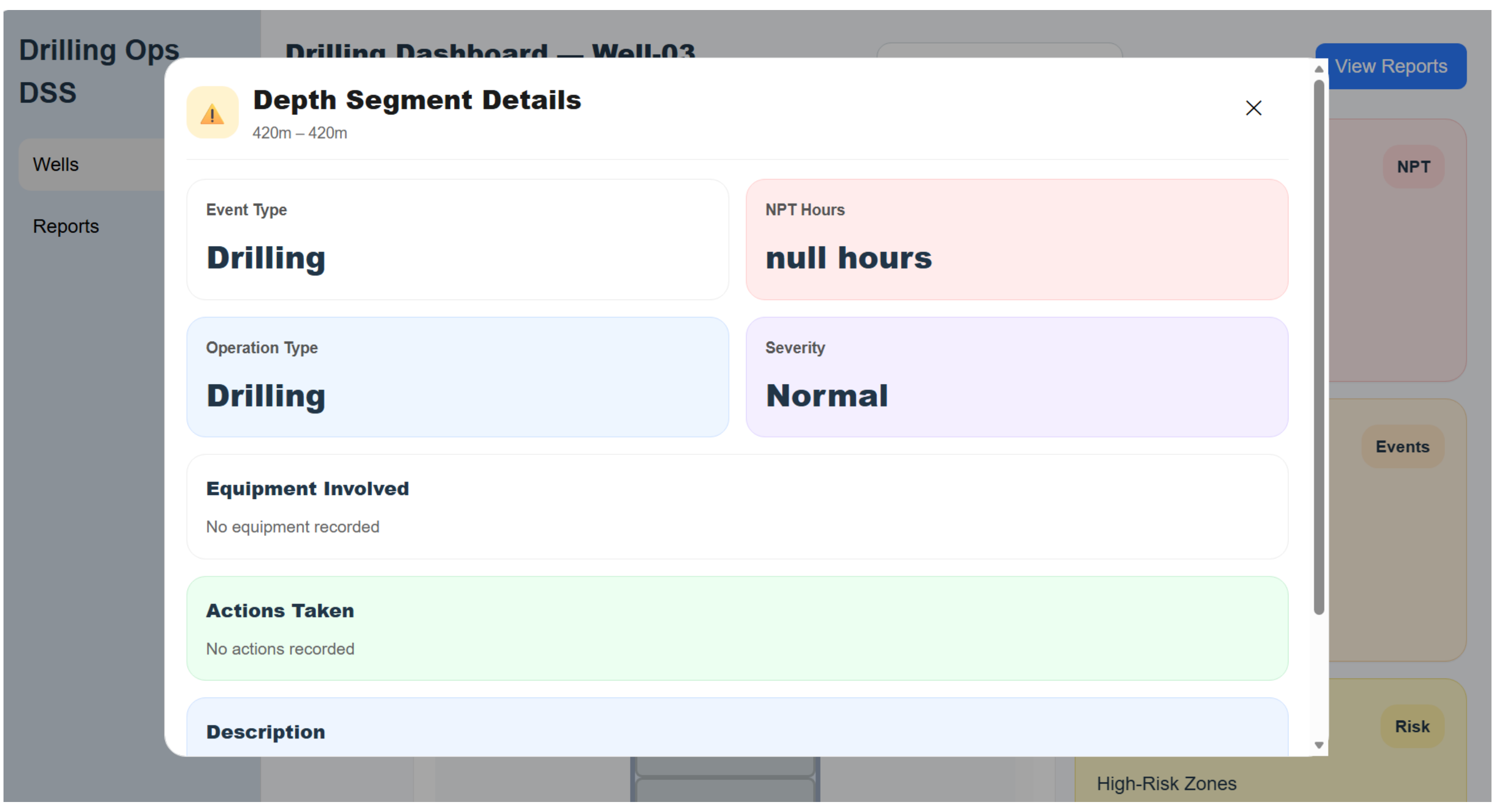


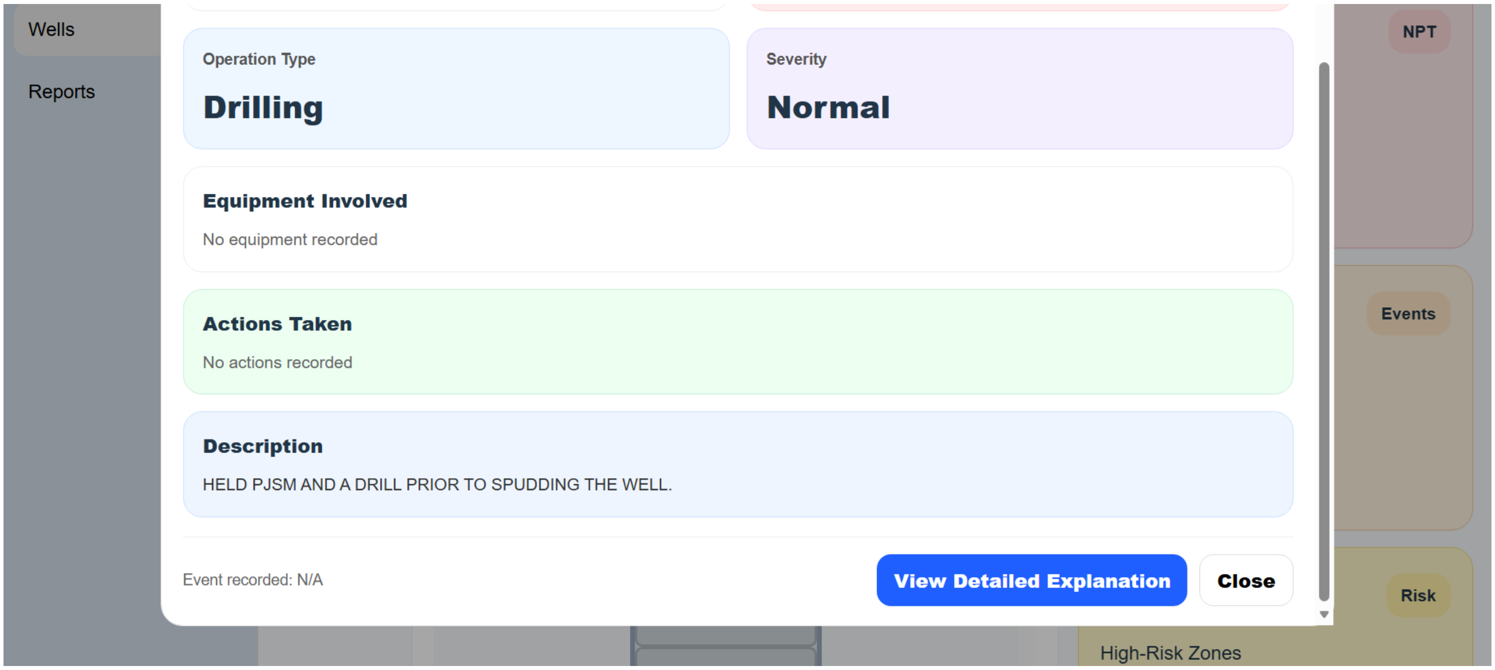
- **Depth-Based Wellbore Visualization**
  Displays the entire wellbore from surface to TD, with each operation represented as a depth-accurate segment. Color coding (normal, warning, critical) helps users instantly identify operational issues such as pack-off, stuck pipe, or high drag zones.
- **Automated Event Classification**
  Each segment is automatically analyzed using rule-based logic to determine severity. This allows the system to highlight abnormal drilling behavior without requiring manual tagging.
- **Non-Productive Time (NPT) Tracking**
  Aggregates NPT hours across all operations and presents the total in a dedicated KPI card. This gives engineers a quick understanding of downtime impact and operational efficiency.
- **Event Count & Severity Summary**
  Shows the total number of parsed events and how many of them are classified as critical. This helps users gauge the overall health of the drilling operation at a glance.
- **High-Risk Zone Identification**
  Highlights depth intervals where repeated warnings or critical events occur. This supports proactive decision-making by drawing attention to problematic sections of the well.
- **Predictive Maintenance Access**

Provides a direct link to the predictive maintenance module, where the system evaluates operational patterns to forecast potential equipment failures or drilling hazards.

- **Report Navigation**
  A quick-access button allows users to jump directly to the list of uploaded daily reports for further review or auditing.
- **Live Data Integration**
  The dashboard pulls real-time structured data generated from PDF ingestion, ensuring that the visualization and KPIs always reflect the most recent drilling activity.
- **Clean, Engineer-Focused Layout**
  The interface is designed to reduce cognitive load—depth markers, color-coded segments, and KPI cards make complex drilling data easy to interpret at a glance.

5.7 Segment Details Panel



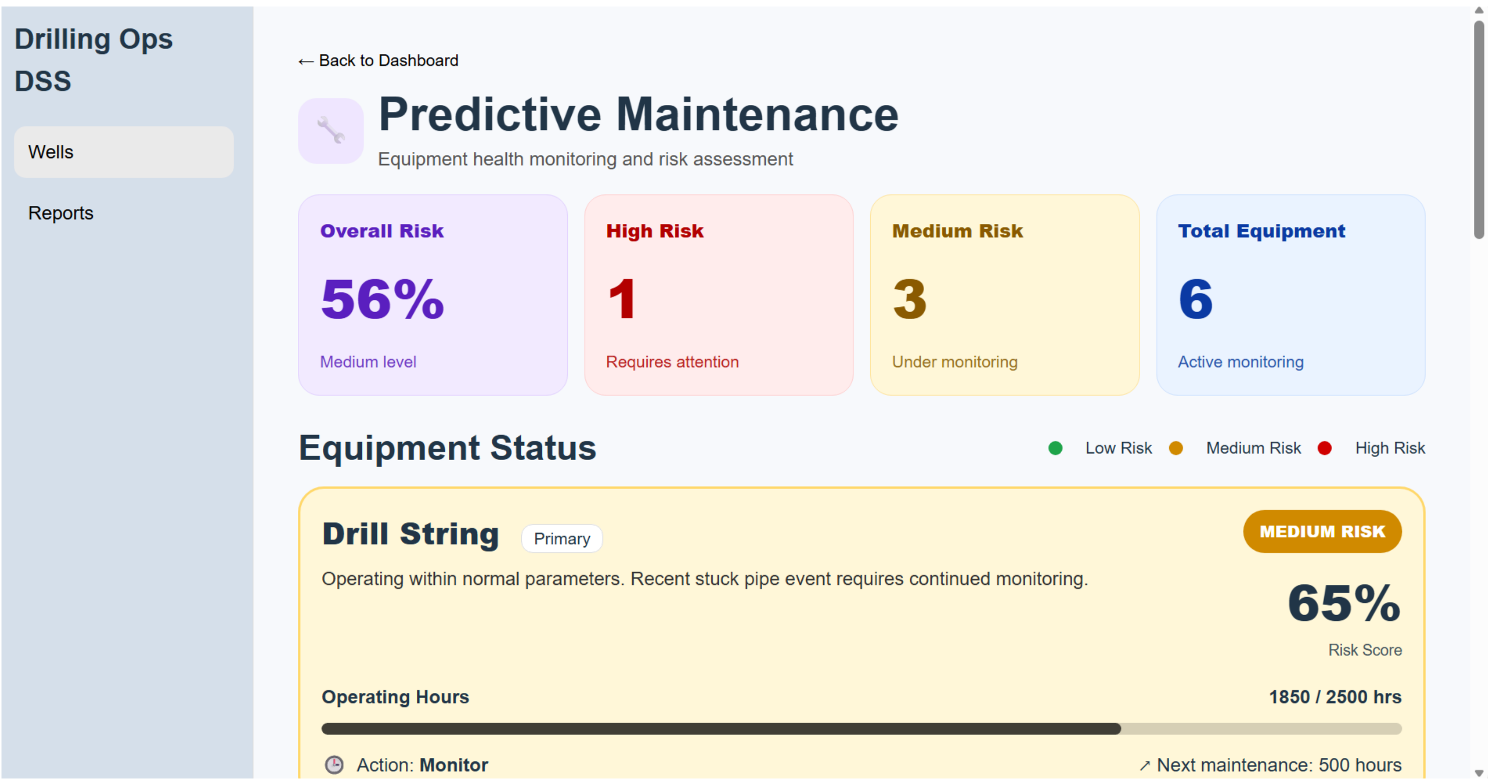


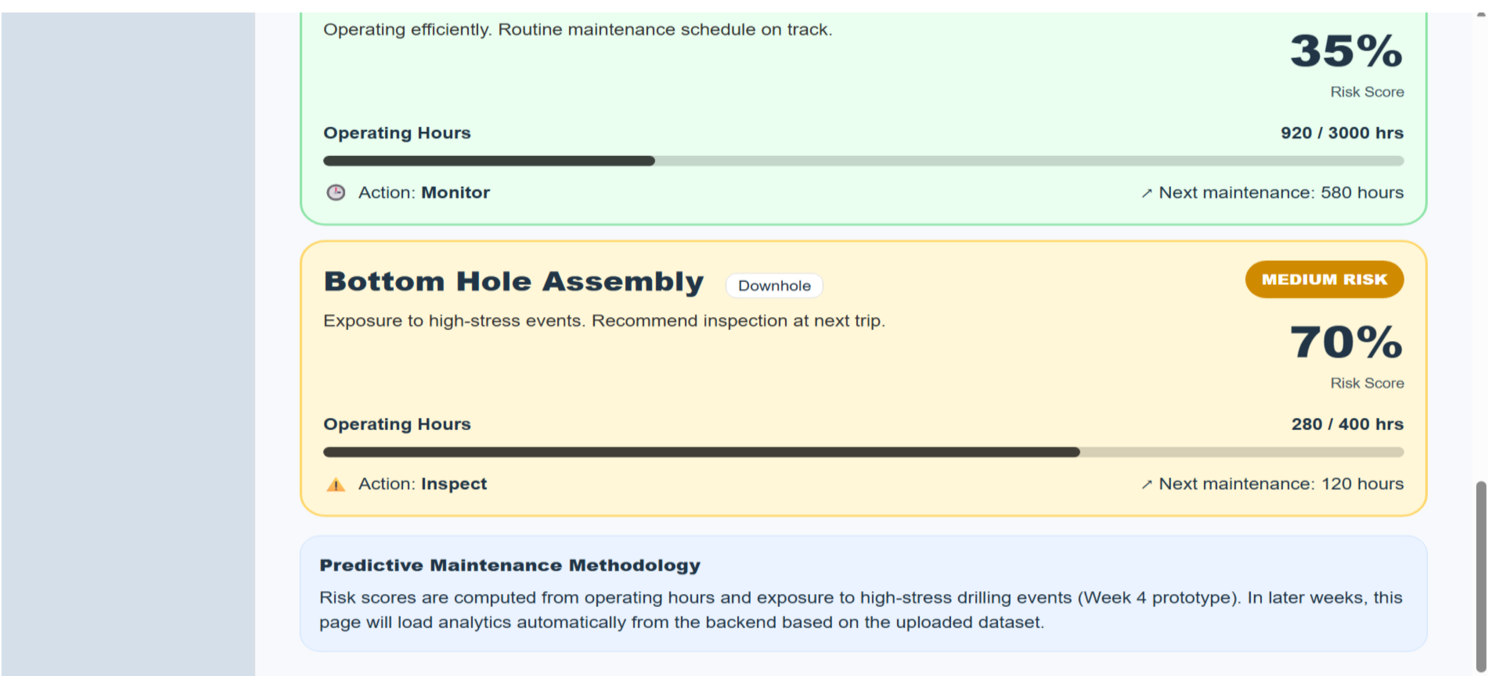
- **Depth-Specific Event Summary**
  This panel displays detailed information for a selected depth interval in the wellbore. It allows engineers to inspect exactly what happened at a specific measured depth, improving clarity when reviewing drilling events.
- **Operation Type Identification**
  Shows the type of operation performed (e.g., Drilling, Circulation, Tripping). This helps users understand the context of the activity recorded at that depth.
- **Severity Classification**
  Indicates whether the event is Normal, Warning, or Critical based on automated rule-based analysis. This classification helps users quickly assess operational risk and identify zones requiring attention.
- **NPT (Non-Productive Time) Tracking**
  Displays the number of NPT hours associated with the selected event. This supports performance evaluation and helps quantify downtime at specific depths.
- **Equipment Involved**
  Lists any equipment referenced in the report for that operation. If none is recorded, the system clearly states this, ensuring transparency in the extracted data.
- **Actions Taken**
  Shows any corrective or operational actions documented in the report. This helps users understand how issues were addressed and whether further intervention may be needed.
- **Event Description**
  Provides the raw text extracted from the PDF report for that depth interval. This preserves the original

operational narrative and supports deeper interpretation of the event.

- **Additional Controls (View Explanation / Close)**
  Users can open a more detailed explanation of the event or close the panel to return to the dashboard. This keeps the interface clean while still offering deeper insights when needed.

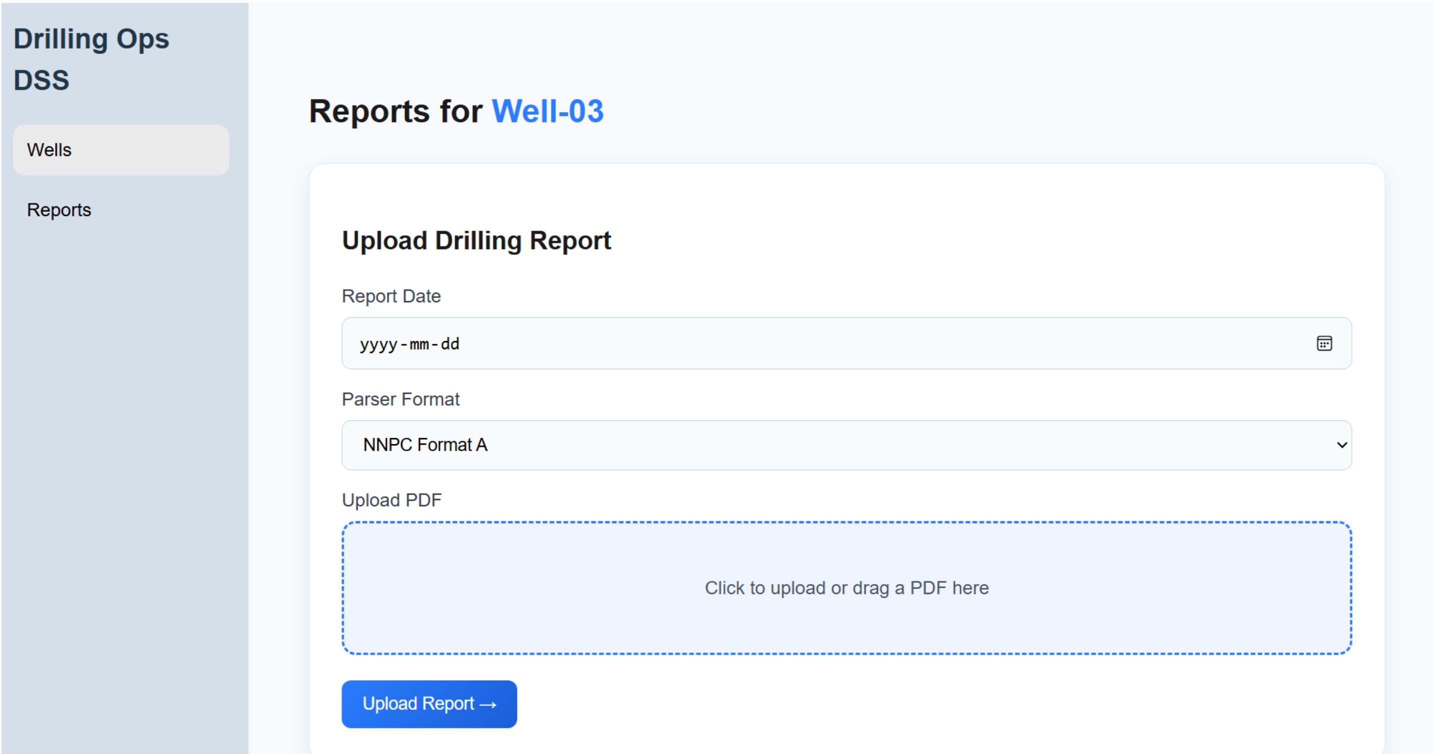
5.8 Predictive Maintenance Page





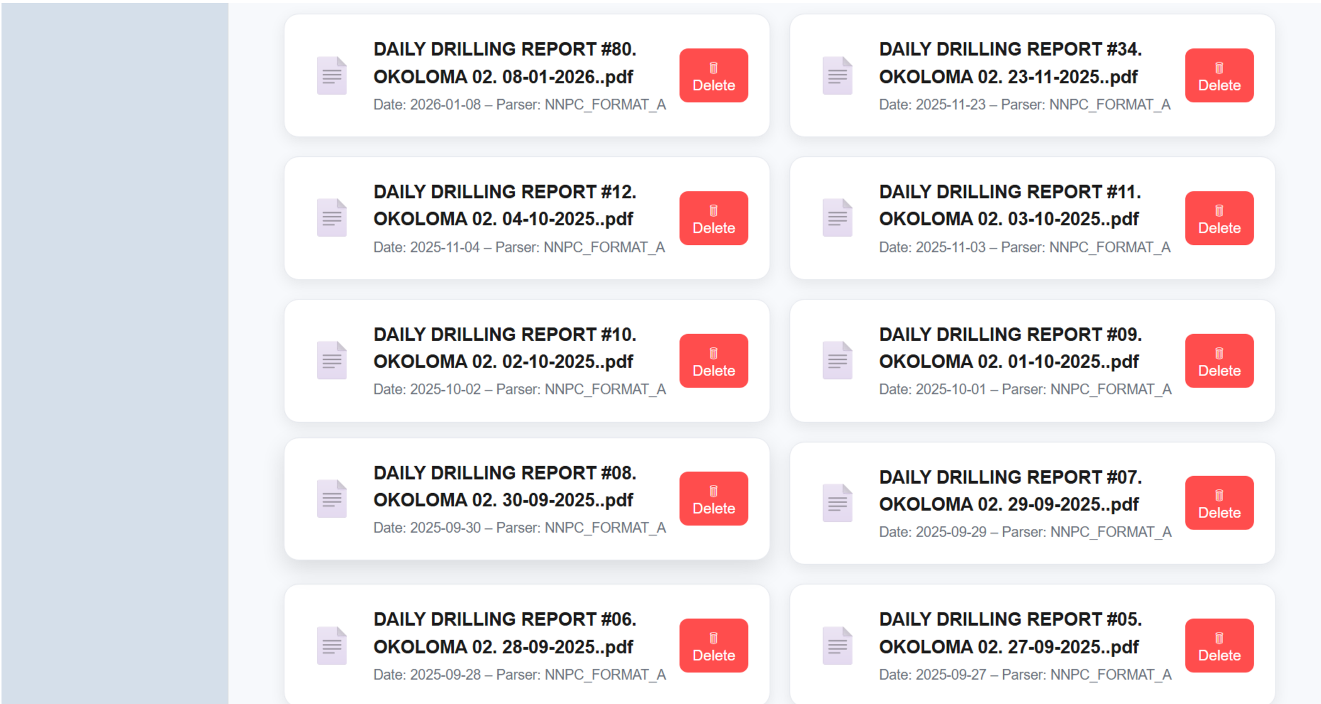
- **Overall Equipment Risk Score**
  Displays a single, aggregated risk percentage representing the combined health of all monitored equipment. This gives engineers an immediate sense of operational safety and potential failure likelihood across the entire system.
- **High-Risk and Medium-Risk Equipment Counters**
  Shows how many components fall into high-risk or medium-risk categories. This helps users prioritize attention, ensuring that critical equipment is addressed before it leads to downtime or safety issues.
- **Total Equipment Under Monitoring**
  Indicates the number of equipment items currently tracked by the system. This provides visibility into the scope of monitoring and ensures all essential components are accounted for.
- **Color-Coded Equipment Status Legend**
  Uses intuitive color markers (green, yellow, red) to represent low, medium, and high risk. This visual cue allows users to quickly interpret equipment health without reading detailed text.
- **Contextual Operational Notes**
  Each equipment card includes a short explanation describing why the component has its current risk score—for example, exposure to high-stress events or recent drilling anomalies. This helps users understand the underlying cause of risk, not just the number.
- **Maintenance Recommendations**
  The system provides actionable guidance such as “Monitor,” “Inspect,” or “Service,” helping engineers plan interventions proactively rather than reactively.
- **Predictive Maintenance Methodology Section**
  Explains how risk scores are calculated—based on operating hours, stress exposure, and event history. This transparency helps users trust the system’s recommendations and understand the logic behind the analytics.

5.9 Drilling Report Page



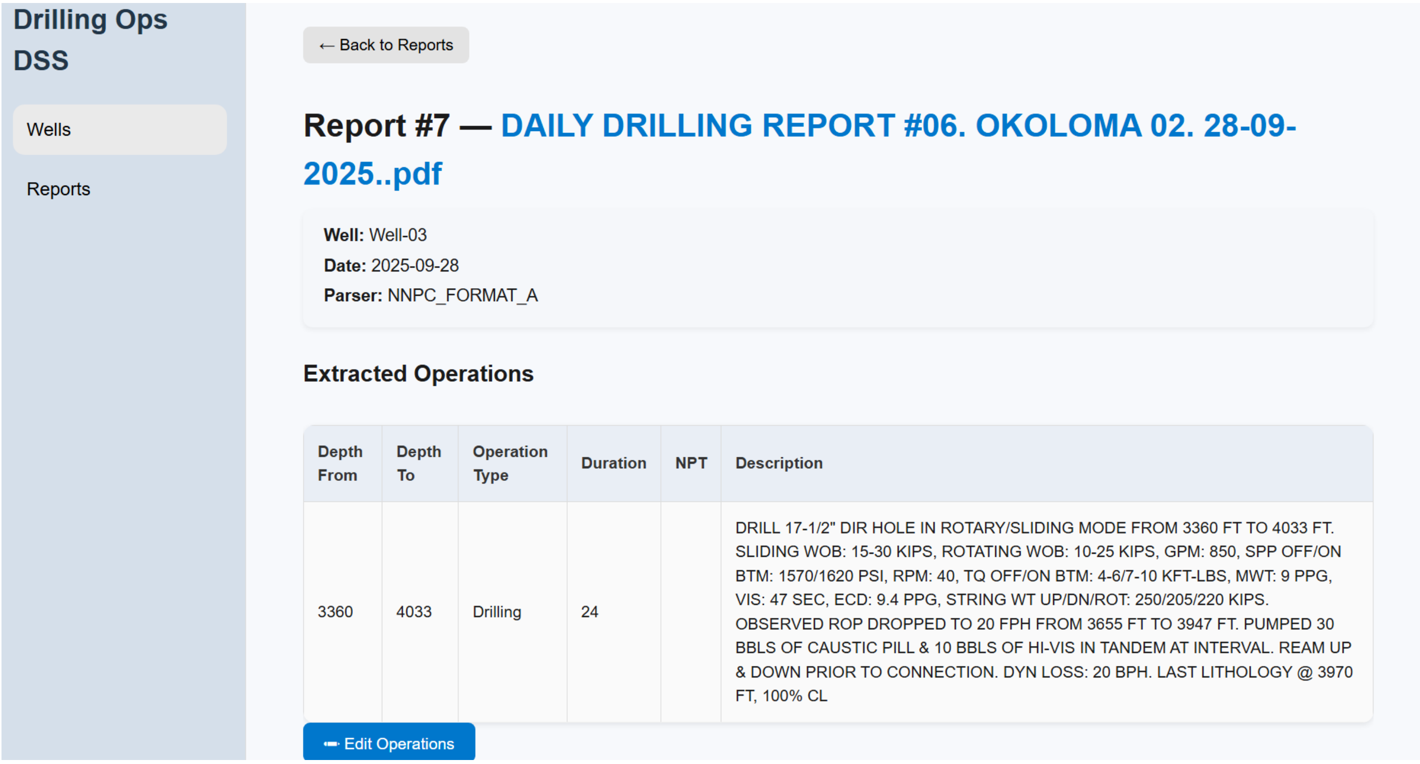
- **Report Date Input**
  Allows the user to specify the exact date of the drilling report being uploaded. This ensures that each PDF is correctly tied to the operational timeline of the well, enabling accurate sequencing, filtering, and historical analysis.
- **Parser Format Selection**
  Provides a dropdown for choosing the correct parsing template (e.g., *NNPC Format A*). Since different operators and rigs use different reporting structures, this option ensures the system applies the right extraction logic for each PDF, improving accuracy and reducing parsing errors.
- **Drag-and-Drop PDF Upload**
  A user-friendly upload area where users can click or drag a PDF file. This is the core entry point for data ingestion — once uploaded, the system automatically extracts depth intervals, operations, NPT hours, and descriptions from the report.
- **Upload Confirmation Button**
  The **Upload Report** → button triggers the backend ingestion pipeline. It validates the file, applies the selected parser, and stores the extracted operations in the database for visualization and analysis across the dashboard.
- **Well Context Header**
  The page clearly displays the current well (e.g., *Well-03*), ensuring users always know which asset they are uploading data for reducing mistakes and improving workflow clarity.

5.10 Daily Reports Page



- **Organized List of Uploaded Daily Drilling Reports**
  Displays all reports associated with the selected well in chronological order. Each entry includes the report name, date, and parser format, giving users a clear overview of the dataset available for analysis.
- **Parser Format Visibility**
  Shows which parsing template (e.g., *NNPC_FORMAT_A*) was used for each report. This ensures transparency in how each PDF was processed and helps users confirm that the correct extraction logic was applied.
- **Delete Report Functionality**
  Each report has a dedicated Delete button, allowing users to remove outdated or incorrect uploads. This keeps the dataset clean and ensures only valid reports contribute to analytics.

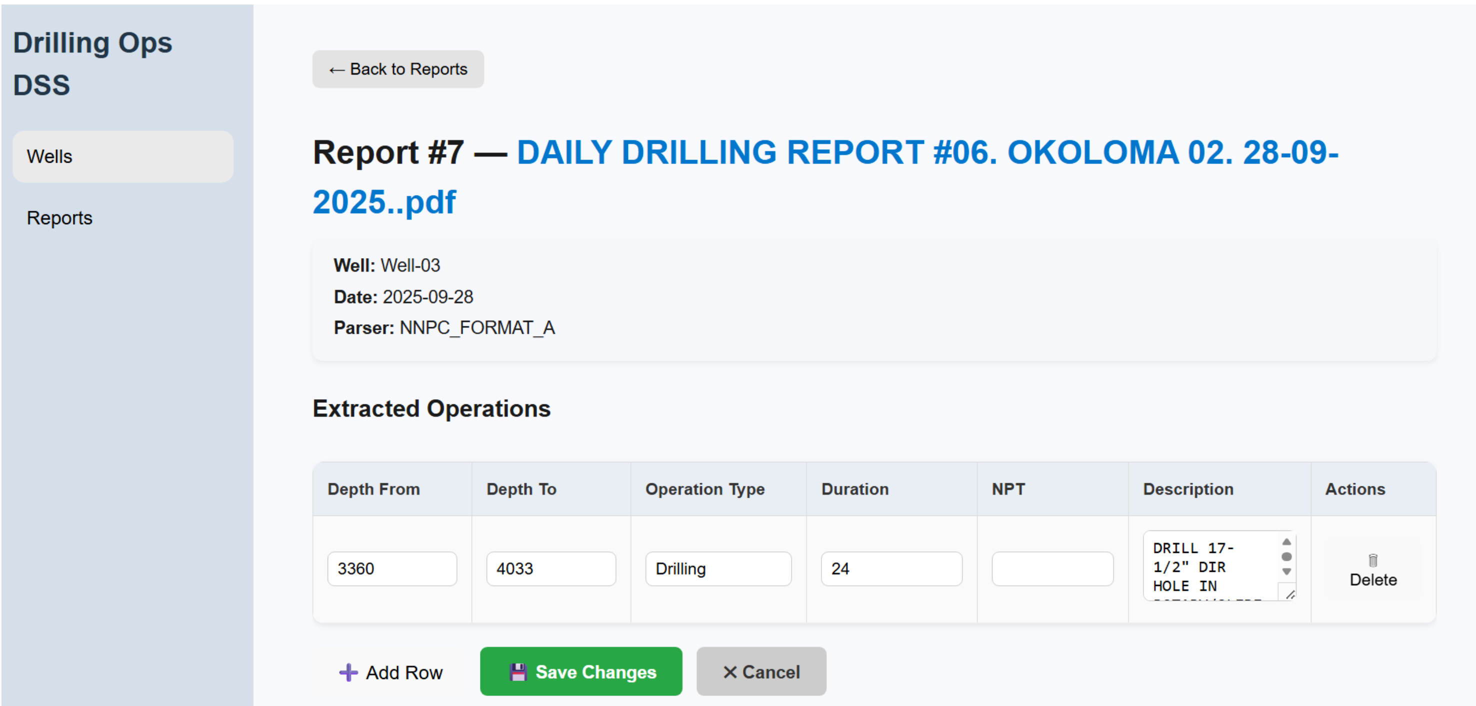
5.11 Extracted Operations Page



- **Parsed Report Metadata**
  At the top of the page, the system displays key information about the uploaded report, including the report number, file name, well name, report date, and parser format. This ensures full traceability and helps users verify that the correct report was processed.
- **Structured Operations Table**
  The core of the page is a table that lists every operation extracted from the PDF. Each row represents a depth-based activity captured in the daily drilling report, converted into clean, structured data for analysis.
- **Depth Interval Extraction**
  The table includes Depth from and Depth To columns, showing the exact measured depth range for each operation. This allows engineers to correlate activities with specific sections of the wellbore.
- **Operation Type Classification**
  Each operation is automatically categorized (e.g., Drilling, Circulation, Tripping). This classification helps users quickly understand the nature of the activity without reading long descriptions.
- **Duration and NPT Identification**
  The system extracts the Duration of each operation and any associated Non-Productive Time (NPT). This supports performance evaluation and helps quantify operational efficiency.
- **Full Description Extraction**
  The Description column contains the raw operational narrative pulled directly from the PDF. This preserves

all engineering details—such as WOB, RPM, GPM, torque, mud properties, and lithology—ensuring no information is lost during parsing.

5.12Extracted Operations Page



- **Editable Operations Table** Shows all parsed operations from the uploaded report, with fields like depth, operation type, duration, NPT, and description.
- **Inline Editing Tools** Users can modify extracted values, correct parsing errors, or refine descriptions directly in the table.
- **Add Row / Delete Row Controls** Allows adding missing operations or removing incorrect ones to keep the dataset accurate.
- **Save & Cancel Actions** "Save Changes" updates the backend with edited data, while "Cancel" discards modifications and restores the original parsed values.

AI Use Section

6.1 AI use Table

AI Tool	Version / Account	Specific Use	Value Added
ChatGPT	GPT-5.2	Assisted with refining the project idea, structuring the proposal, improving academic writing, and clarifying the research design, methodology, and timeline.	Helped organize thoughts clearly, improve academic clarity, and ensure the project scope and structure align with the course.
Copilot	Free	Summary of Relevant Literature and Existing Research	Helped to clearly identify the existing research on the project and showed how the project differs from various working projects.
ChatGPT	GPT-5.2	Used as learning support while developing the React frontend, including understanding React components, JSX structure, routing with React Router, file organization, and troubleshooting common setup and import issues during implementation.	Supported faster understanding of React concepts, helped resolve configuration and import errors, and improved confidence in implementing the frontend correctly within the project timeline.
Figma AI	Free	Used to generate UI layout suggestions and design inspiration for the drilling dashboard, including card layout, spacing, and visual hierarchy.	Helped to clearly identify the existing research on the project and showed how the project differs from various working projects.
Copilot (Backend Development)	Free	Assisted with setting up the FastAPI backend, configuring SQLite, creating database models, organizing folder structure, and planning the data-upload workflow.	Accelerated backend setup, ensured clean architecture, and improved clarity on how data flows from upload → database → analytics → frontend.

Copilot (Backend Development)	Free	I worked through the Excel upload pipeline, including column normalization, validation, duplicate handling, row-level error reporting, and debugging the upload endpoint.	It helped me break down the errors clearly, understand what was causing the failures in the upload pipeline, and refine the normalization and validation flow so the backend is more stable, predictable, and better prepared for the analytics features I'll build next.
Copilot (Backend & Frontend Development)	Free	Help design and refine multiple core features across the system, including the Login and Logout pages, authentication flow, dashboard layout, and React routing structure. Assisted in debugging CSS layout issues, fixing sidebar overlaps, centering authentication screens, and improving the UI for the wellbore visualization, predictive maintenance cards, and report upload forms. Supported backend development by helping structure FastAPI endpoints, compute KPIs, design the /segments and /dashboard routes, and clean up the data flow from PDF parsing frontend visualization.	My development speed significantly improved. I was able to solve layout bugs faster, structure my backend more cleanly, and build a polished, professional UI without getting stuck on small issues. Copilot helped me think through design decisions, reduce errors, and maintain a consistent architecture across the entire project. It made complex features—like the wellbore visualization and predictive maintenance module—much easier to implement and refine.

Work Date / Hours Log

Date	Hours	Description
17-Jan-2026	3.5	- I explored and researched possible project ideas for the course, focusing on areas related to drilling operations and data analytics. I evaluated whether the project idea was feasible to implement within the course timeline and researched whether similar work had been done before. This helped me narrow down and decide on a suitable project that aligns with my background and course requirements.
20-Jan-2026	3	- I started learning React by reviewing the basics of components, JSX, and how a React application is structured. I focused on understanding how React will be used to build the frontend of my applied project.
21-Jan-2026	3.5	- I worked on Introduction of the project: Improving the introduction, stating the problem statement clearly, outlining what questions needs to be answered. - I searched for literature and research work based on my project to find similar research on my project. I searched for gaps within this research project work.
22-Jan-2026	2.5	- I continued literature research related to drilling operations, nonproductive time, and decision support systems. I reviewed how similar systems visualize operational data and noted limitations in existing approaches, particularly depth-based visualization. - I worked on my reference and included it in my project report after the research I did.
23-Jan-2026	3	- I worked on defining the project methodology, including data sources, data collection approach, and analysis techniques. - I also clarified how simulated drilling data would be used and documented the justification for using non-proprietary datasets in the project.
25-Jan-2026	1	- I created the project timeline for the complete project for the entire semester.

26-Jan-2026	3.5	- I drew the Gantt chart for the project timeline and included it in my proposal report. - I prepared the appendix. - I went through the entire report and did a lot of corrections and restructuring to make sure the proposal fits what my project is all about.
27-Jan-2026	2	- I set up the project GitHub repository structure according to the course requirements and organized folders for implementation and documentation. - I created the frontend, backend, and data directories under the implementation folder to support a clean development workflow.
29-Jan-2026	5	- I initialized the React frontend using Vite within the repository and verified that the development server runs correctly on my local machine. This step confirmed that the frontend environment was properly configured and ready for further development. - I continued learning React by implementing the initial application layout and navigation structure. I worked on creating reusable components for the application frame (layout and navigation) and began organizing the project into pages and components to follow best practices. I separated the HTML structure (JSX) and styling (CSS) into different files to improve readability and maintainability. I also installed and configured React Router to support page navigation between the wells list, dashboard, and reports sections of the application. - I implemented the well selection page and routing logic to allow navigation between different wells and their corresponding dashboard views. I tested navigation behavior to ensure that selecting a well correctly loads the appropriate dashboard page. During this process, I encountered and resolved common React

		development issues, including file naming inconsistencies, incorrect import paths, and component casing errors. I fixed these issues by standardizing file names, correcting relative import paths, and restarting the development server as needed, ensuring the application builds and runs successfully after all fixes.
02-Feb-2026	3.5	- -Built and refined Wells page with color-coded well status cards - Implemented depth-based wellbore visualization - Added clickable depth segments with event details modal - Integrated routing between wells and dashboards - Improved frontend layout and interaction flow
05-Feb-2026	4	- I set up flexible column normalization so the upload can handle different Excel header names - I added validation for wells, operations, and events during the upload process - I added row-level error reporting to make debugging easier and catch bad data - I improved the duplicate checks so existing records aren't inserted - I added logging across the upload flow for better visibility - I finished wiring up the /upload endpoint with multipart file support - I updated the utils for mapping, normalization, and safer data parsing - I prepared the backend for future analytics by making sure uploaded data is clean and consistent

06-Feb-2026	2	- I created simulated datasets and collected sample datasets from different companies so I could compare how each one structures its drilling data. - I analyzed the differences in column names, formats, units, timestamps, and category labels across these datasets.
09-Feb-2026	5	- Implemented a complete rewrite of the Excel upload pipeline for wells, operations, and events. - Fixed all column-name mismatches between the transformer output and SQLAlchemy models. - Updated the Well insertion logic to use well_name instead of the invalid name argument. - Corrected the operations insertion to use operation_type instead of the non-existent op_type. - Updated event insertion and duplicate-check logic to use the correct model field (event_time). - I rebuilt the transform_dataset() function to ensure consistent normalization, depth conversion, timestamp parsing, and category mapping. - Ensured all transformed sheets contain the required columns before insertion. - Improved error reporting for wells, operations, and events to make debugging easier. - Cleaned up duplicate imports and removed unused or inconsistent logic. - Stabilized the entire ETL flow so uploads now run end-to-end with predictable behavior. -

13-Feb-2026	5	- Created operations router - Added GET /wells/{well_id}/operations endpoint - Added GET /wells/{well_id}/segments endpoint - Converted database operations into frontend-ready wellbore segments - Implemented basic risk-level classification logic - Connected router to main.py - Implemented DailyReport model and ingestion workflow for PDF uploads - Added duplicate detection using file hash and (well_id + report_date) - Created NNPC_FORMAT_A parser using pdfplumber with table-based extraction - Extracted operation rows from Operation Summary section - Implemented depth extraction logic (MD from / MD to detection) - Linked parsed operations to DailyReport and Well - Added debugging preview fields (matched_rows_preview, debug_preview). - Backend now supports: - Creating wells - Uploading daily drilling PDF reports - Preventing duplicate uploads - Parsing operation summary rows into database - Retrieving operations via GET endpoint
14-Feb-2026	4	- -Added CORS + created /wells/{well_id}/dashboard endpoint and updated React Dashboards.jsx to fetch real segments/KPIs instead of hardcoded data. - Cleaned upload router to only handle PDF ingestion and moved wells/operations/segments endpoints under /wells routes to avoid duplicate Swagger entries. - Created wells router with create and list endpoints. Cleaned upload router to handle only report uploads.

		- Added CORS + created /wells/{well_id}/dashboard endpoint and updated React Dashboards.jsx to fetch real segments/KPIs instead of hardcoded data. - Fetch wells list from backend /wells/ Clicking a well navigates to /wells/:wellId dashboard
15-Feb-2026	4.5	- Added /reports router with list and detail endpoints - Enabled fetching all uploaded reports from DB - Enabled viewing a single report with its parsed operations - Registered router in main.py - Backend now supports a full Reports Page UI - Re-enable KPI light tone styling via tone classes Convert wellbore into fixed-height scroll view Add depth cursor slider + merge contiguous normal segments for cleaner visualization. - Implemented Reports.jsx to fetch /reports from backend - Added ReportRow.jsx component for list UI - Added Reports.css styling - Enabled navigation to individual report pages - Frontend now displays all uploaded drilling reports dynamically
16-Feb-2026	4	- Implemented a dedicated UploadReport.jsx page with styled form for uploading PDF reports. - Added UploadReport route in App.jsx and integrated upload button at the well level. - Added modal-based Create Well form directly inside Wells.jsx for quick well creation. - Redesigned Report Detail page to use a professional table view instead of cards. - Added new CSS: UploadReport.css and ReportDetail.css for a clean UI experience. - I Connected all frontend pages with backend routes properly. - Prepared UI structure for full multi-well report categorization.
19-Feb-2026	6	- Updated upload_daily-report FastAPI endpoint to use Form(...)

		for well_id, report_date, and parser_type. This ensures the frontend can upload reports using multipart/form-data. - Fixed 422 validation errors and restored successful PDF ingestion. - Added DELETE /reports/{report_id} endpoint in FastAPI - Added delete button to each report card in Reports.jsx - Added confirmation dialog and optimistic UI update after deletion - Added new CSS styles for delete button and improved card layout - Ensured seamless integration with existing upload and report detail flows. - fix(reports): register DELETE endpoint correctly - Moved delete_report() outside get_report_details() - Ensures FastAPI registers DELETE /reports/{report_id} - Resolves 405 Method Not Allowed during report deletion - main(wells): add Recently Viewed wells + Quick Actions bar + UI improvements - Added Quick Actions bar (Create Well, View Reports, Upload Report) - Added stats bar and modernized layout - Added color-coded well cards, operator avatars, and status badges - Improved overall dashboard experience - fix(wells): correct quick-action routing and add well-selection modal - View Reports and Upload Report now open a modal to choose a well - Added divider line to separate quick actions from wells - Improved navigation logic and UX consistency - style(wells): redesign Create Well modal for improved UX

		- Added modern modal card with animations - Improved input styling and labels - Added structured footer with Create + Cancel buttons - Enhanced spacing, shadows, and visual hierarchy
20-Feb-2026	4	- implement Sign Up page and full localStorage-based authentication - Added SignUp.jsx and SignUp.css - Updated Login.jsx to validate against stored user accounts - Added /signup route and integrated with login flow - Enabled full sign-up → login → dashboard experienc - added Sign Up link to Login page and complete auth flow - Added "Don't have an account? Create one" link - Linked Login → SignUp route - Updated styling for login switch link - Ensured smooth SignUp → Login → Wells navigation - Moved login/signup routes outside Layout wrapper - Added premium gradient background for auth pages - Optional glassmorphism card styling for modern UI
21-Feb-2026	2.5	- Replaced old icon with a cleaner well-bore-style SVG - Added new Home.css with modern gradient background and centered content - Updated Home.jsx to use new styling and improved structure - Ensured Home page loads without sidebar layout - Refined CTA button and feature pills for a more polished landing experience

23-Feb-2026	2	- Updated global App.css to ensure html, body, and #root all have width: 100% and height: 100% Prevented layout shrink issues caused by flex containers higher in the component tree Ensured that routed pages (e.g., Wells, Reports) now correctly expand to full available width Improved overall stability of the layout across all pages rendered inside <Layout/>

Conclusion

This project successfully delivered a Drilling Operations Decision Support System that transforms unstructured daily drilling reports into actionable intelligence. Through automated PDF parsing, structured data storage, depth-based wellbore visualization, KPI dashboards, and a predictive maintenance module, the system provides engineers with a clear, data-driven view of well performance. The development process integrated both backend and frontend engineering, including authentication flows, report management, interactive dashboards, and editable operation tables. By combining clean architecture with intuitive UI design, the platform demonstrates how modern data engineering and analytics can significantly enhance drilling oversight, reduce manual workload, and support proactive decision-making. Overall, the project showcases a scalable foundation for future enhancements such as advanced machine learning models, multi-well analytics, and real-time operational monitoring.

References:

Literature references

WellVizion - Well schematics diagramming solution. (n.d.). https://wellvizion.com/

Admin. (n.d.). *PetrolInk: Data Visualization in Drilling Operations – Petrolink*. https://www.petrolink.com/data-visualization-in-drilling/

HalVue^{TM} real-time viewer. (n.d.). Halliburton. https://www.halliburton.com/en/products/halvue-real-time-viewer

Barakat, M., Ela, M. a. E., & Khalaf, F. (2021). Integrating risk management concepts into the drilling non-productive time. *Journal of Petroleum Exploration and Production Technology*, *11*(2), 887–900. https://doi.org/10.1007/s13202-020-01059-0

Liu, S., Feng, Y., Wang, X., & Yan, P. (2021). Cross-Platform Drilling 3D Visualization system based on WebGL. *Mathematical Problems in Engineering*, *2021*, 1–18. https://doi.org/10.1155/2021/5516278

Corva.Ai. (n.d.). *AI Predictive Drilling Automation Software | Corva*. Corva. https://www.corva.ai/energy/predictive-drilling

Appendix

AI Prompt(s) Used

- Explain React components, pages, layout, and routing like I'm a beginner, and guide me step-by-step to set up a React app using Vite on Windows.
- I'm building a drilling dashboard UI. Help me create a Week 4 React layout with wells list + navigation. Keep CSS in separate files and explain what each file does.
- I'm getting an import/casing error in Vite/React. Diagnose the error and tell me exactly what files to rename and what import paths to use.
- How do I modify the frontend such that when I click on the view detailed explanation, it goes ahead to show me the detailed explanation and the analytics gotten based on the data provided? Please also explain very well so I can understand how it is done.
- No there is already <div className="footerBtns"> <button className="primaryBtn" type="button"> View Detailed Explanation </button> <button className="secondaryBtn" type="button" onClick={onClose}> Close </button> </div> so when you chick on the button that's where the three pictures come in.
- Okay, when I put the backend it will generate the analytics on its own without putting any explanations
- I want to put a button to show back to reports after uploading it is successful. And also, the depth is not working again it did not capture the dept correctly. Also, i would like us to put a full designed upload on the report side because the current upload is when you click on the wells and go to view report. but this time i want it directly on the report page under the wells page in the component part.
- so my wellbore pipe segment is meant to show different color codes when it detects that the report that was uploaded has any critical word in it, in the description but it is not showing

Microsoft Copilot

- let us work based on my project. do I need the data to start so that I can do some analytics to connect it to the front end? or let us start with , obviously you upload your dataset, then the app should receive this dataset and analyze it and show it on the wells page. When you then click on the particular well, it will show you the drilling pipe with different color coding. But first we should work on uploading the dataset first.
- For the uploading of files, these 3 categories, are you uploading the three documents as one csv, that is the; wells, operations, events. Also, most of this dataset have to be

prepared after drilling right? that means there has to be some data input into their various tables. Or can the dataset from the company have everything, then the app begins to separate it by itself. That is to say, does it have to be prepared already.

- Good now, here in my wells, it looks so blank apparts from the wells that was created that is showing. What else do you think i can put here in the wells part to make it look more informative rather than showing only wells.
- when you click view report and upload report it shows page not found. can you link it to the exixting ones also move the quick bar to the side or draw a line to differenttiate it from the wells. Also i was thinking when you click on view reports you have to select the well first you want before seeing the reports. while upload report when you click on it, it can take us to the one we have already created.
- Okay now i see why the color codes are not showing on the segment. I tried editing one of the decription and i put pack-off inside so that it can display the color code. But i noticed the edit is not updated in the segment but it updated in the report details and backend. so the in the segment if you click the segment modal for each block it shows the original description but doesnt show the one i edited. so my testing did not work. Do you understand what i am trying to explain. so the segment is getting the description dirrectly from the uploaded report not minding if it was edited or not so it doesnt show my update. I edited it cause i just wanted to test the colors on the wellbore. Do you understand? and what can i do now?
- lets continue the upload i think most companies use excel not csv right so is it better to use excel to upload the data or csv based on what companies use.
- I dont want the exact names because not everyone in the industry would have time to put the exact names.
- okay if I integrate the script into the endpoint upload, that means it is ready for use. what does that mean. Also, after integrating it, can i test it to see if it is working. secondly will the upload be able to carry thousands of roles of data